

ÜBUNG 3

Räumliches Echtzeit-Audio im Kontext von Computerspielen

Abgabe

Narek Grigoryan Mtr.Nr : 1547616

15. September 2025

Inhaltsverzeichnis

1	Ziel der Übung	2
2	Definition und theoretische Anforderungen von räumlichem Echtzeit-Audio	2
3.	Verfahren zur räumlichen Audioverarbeitung und ihre Komplexität	3
3.1	HRTF-basiertes binaurales Rendering	3
3.2	Ambisonics: First- und Higher-Order Formate	4
3.3	Bildquellen-Methode und Ray-/Path-Tracing	5
3.4	Geometrie-basiertes Delay- und Attenuation-Routing (Tapped-Delay-Line)	6
4.	Diskussion der Trade-offs	7
4	Engines und Middleware: Praxis-Check	8
4.1	Unity (Fokus)	8
4.1.1	Architektur und Verfahren	8
4.1.2	API-Umfang (Unity-Sicht)	9
4.1.3	Lizenzmodell, Portabilität, typische Performance	9
4.2	Godot (kurz)	10
4.3	Unreal Engine (kurz)	10
5	Gedankenexperiment: Implementierungsskizze für mein Labyrinth (Unity auf macOS)	10

1 Ziel der Übung

Die dritte Übung im Rahmen des Projekts widmet sich dem Themengebiet des räumlichen Echtzeit-Audios (Spatial Audio). Während in den ersten beiden Übungen vor allem die visuelle Gestaltung der Spielumgebung sowie die Einbindung einer KI-gesteuerten Spielfigur im Vordergrund standen, richtet sich der Fokus nun auf die akustische Dimension. Ziel ist es, zunächst den Begriff des räumlichen Echtzeit-Audios zu definieren und die dahinterstehenden theoretischen Grundlagen zu beleuchten. Anschließend sollen unterschiedliche technische Verfahren zur Umsetzung von Spatial Audio verglichen und im Hinblick auf ihre zeitliche und speicherbezogene Komplexität analysiert werden. Dabei gilt es auch, die typischen Trade-offs zwischen realistischer Klangqualität und der Rechenlast in einer Echtzeitanwendung wie einem Videospiel herauszuarbeiten. Darüber hinaus ist zu untersuchen, welche gängigen Game Engines oder Audio-Middleware bereits Unterstützung für bestimmte Spatial-Audio-Techniken anbieten und wie sich diese in das bestehende Labyrinth-Projekt integrieren ließen. Die Übung verfolgt somit das Ziel, ein vertieftes Verständnis für die Rolle von Audio in immersiven digitalen Umgebungen zu entwickeln und konkrete Umsetzungsmöglichkeiten für die eigene Spieleentwicklung zu skizzieren.

2 Definition und theoretische Anforderungen von räumlichem Echtzeit-Audio

Unter räumlichem Echtzeit-Audio versteht man die Fähigkeit eines Audiosystems, Klänge so wiederzugeben, dass sie für den Hörer nicht nur in der Ebene von links nach rechts, sondern in einem vollständigen dreidimensionalen Klangfeld lokalisiert werden können. Während klassisches Stereo lediglich die horizontale Richtung vermittelt, ermöglicht Spatial Audio eine exakte Wahrnehmung von vorne, hinten, oben und unten. Der Spieler oder die Spielerin erlebt dadurch eine realistische akustische Umgebung, die das Gefühl vermittelt, tatsächlich in der virtuellen Welt anwesend zu sein [4] [21].

Die Grundlage für diese Immersion bilden verschiedene psychoakustische Mechanismen. Besonders wichtig sind die sogenannten interauralen Laufzeitdifferenzen (ITD), also die winzigen Zeitunterschiede, mit denen ein Schallereignis die beiden Ohren erreicht. Hinzu kommen interaurale Pegeldifferenzen (ILD), die sich daraus ergeben, dass Schallquellen auf einer Seite des Kopfes lauter wahrgenommen werden als auf der gegenüberliegenden. Außerdem spielt die individuelle Form der Ohrmuschel eine große Rolle, da sie den Klang frequenzabhängig filtert und damit Hinweise auf die vertikale Richtung und Tiefe gibt. Diese Effekte werden in sogenannten Head-Related Transfer

Functions (HRTFs) mathematisch modelliert und sind ein zentraler Bestandteil vieler binauraler Verfahren.

Für den Einsatz in Computerspielen ergeben sich daraus besondere Anforderungen [22]. Zum einen muss die gesamte Berechnung in Echtzeit erfolgen, da sich sowohl die Spielfigur als auch andere Objekte kontinuierlich bewegen. Jede Änderung der Kopfhaltung oder Blickrichtung des Spielers muss sofort in der akustischen Darstellung berücksichtigt werden, damit der räumliche Eindruck nicht verfälscht wird. Gerade in Virtual-Reality-Anwendungen ist diese dynamische Anpassung an das Head-Tracking entscheidend, da schon minimale Verzögerungen oder Abweichungen das Gefühl der Präsenz erheblich beeinträchtigen können.

Zum anderen verlangt ein überzeugendes Spatial Audio nach einer Simulation der Raumakustik. Klänge prallen an Wänden ab, werden durch Objekte teilweise abgeschwächt (Obstruction) oder vollständig blockiert (Occlusion), und erzeugen Nachhall (Reverberation), der die Größe und Beschaffenheit eines Raumes vermittelt. Ein Spiel, das ein realistisches akustisches Erlebnis erzeugen möchte, muss diese Phänomene mit Hilfe von Algorithmen für Schallausbreitung, Hallberechnung und Filterung abbilden.

Schließlich spielt auch die technische Komplexität eine zentrale Rolle. Je höher die Präzision der Simulation, desto größer ist der Rechenaufwand. So bieten etwa Verfahren wie Ambisonics mit höheren sphärischen Harmonischen eine feinere Lokalisation, benötigen dafür jedoch deutlich mehr Rechenleistung und Speicherplatz. Ähnliches gilt für aufwendige Faltungsverfahren, die individuelle HRTF-Datensätze nutzen. In der Praxis von Echtzeit-Spielen muss deshalb stets ein Gleichgewicht zwischen der angestrebten akustischen Qualität und den verfügbaren Systemressourcen gefunden werden.

Zusammenfassend lässt sich sagen: Räumliches Echtzeit-Audio erweitert die akustische Dimension von Computerspielen erheblich, stellt Entwicklerinnen und Entwickler aber auch vor die Herausforderung, die komplexen Anforderungen an Psychoakustik, Echtzeitfähigkeit, Raumakustik und Performance in Einklang zu bringen.

3. Verfahren zur räumlichen Audioverarbeitung und ihre Komplexität

3.1 HRTF-basiertes binaurales Rendering

Ein klassischer Ansatz für räumliches Audio am Kopfhörer ist das **binaurale Rendering**. Dabei wird das Signal jeder Schallquelle mit einem „Head-Related Transfer Function“ (HRTF) gefaltet. HRTFs modellieren die Laufzeit-, Pegel- und spektralen Unterschiede, die durch den Kopf, den Körper und insbesondere die Ohrmuschel ent-

stehen [22]. Dadurch kann das Audiosystem den Eindruck erwecken, als käme ein Geräusch aus einer bestimmten Richtung und Entfernung. Allerdings ist diese Faltung rechenintensiv: Das NASA-Technical-Report zur binauralen Simulation räumt ein, dass „die Anwendung der HRTFs **computationally expensive**“ ist und deshalb eine Echtzeit-Verarbeitung vieler Quellen einschränkt. Später im Bericht wird betont, dass die Convolution der HRTF-Filter der Hauptgrund für den hohen Rechenbedarf ist und dass eine Vorverarbeitung wie die **Singular-Value-Decomposition** (SVD) oder die „Equivalent Source Reduction“ den Aufwand mindern soll [11].

Die Zeitkomplexität ergibt sich aus der Filterlänge und der Signallänge. Bei einer **direkten Zeit-Domänen-Faltung** muss für jedes der N Eingangssamples eine Summe über M HRTF-Impulse berechnet werden, was insgesamt $O(N \cdot M)$ bzw. für identische Längen $O(N^2)$ ergibt. In der Literatur zur effizienten Convolution wird dies als „quadratisch“ bezeichnet. Mit **Fast-Fourier-Transform-basierter Faltung** lässt sich der Aufwand auf $O(N \log N)$ reduzieren, indem das Signal und die HRTF in den Frequenzbereich transformiert und dort punktweise multipliziert werden [9]. Dieser Trick verringert die Rechenzeit erheblich, erfordert aber mehr Puffer-Speicher für die Blockverarbeitung. In der Praxis benutzen HRTF-Engines oft Überlapp-Speicherverfahren (overlap-save/overlap-add), um den Speicherbedarf zu kontrollieren. Neben dem reinen Faltungsaufwand müssen auch die Speicherkosten für die HRTF-Bibliothek berücksichtigt werden: Hochwertige Systeme arbeiten mit Hunderten von Impulsantworten für verschiedene Winkel und Entfernungen, was leicht einige Megabyte beansprucht. Manche Forschungsprojekte nutzen SVD oder Principal-Component-Analyse, um die Zahl der Koeffizienten zu reduzieren.

Der **Trade-off zwischen Präzision und Laufzeit** zeigt sich in der Filterlänge: kurze, grob gemessene HRTF-Impulse benötigen wenig Rechenzeit, führen aber zu einem verwaschenen oder unscharfen räumlichen Eindruck. Längere Impulsantworten (z. B. 512–2048 Samples) erfassen subtile Pinna-Resonanzen und liefern eine bessere Höhen- und Tiefenortung, erhöhen jedoch den Faltungsaufwand. Hinzu kommt, dass generische HRTF-Datensätze nicht auf jedes Ohr passen; personalisierte HRTFs verbessern die Lokalisationspräzision, erfordern aber einen höheren Mess- und Speicheraufwand. In interaktiven Spielen wird daher häufig ein Mittelweg gewählt: kurze, vorgefilterte HRTFs, teilweise per FFT beschleunigt, und ggf. Cross-fading zwischen mehreren Datensätzen für bewegte Quellen.

3.2 Ambisonics: First- und Higher-Order Formate

Ambisonics kodiert ein Schallfeld nicht als einzelne Kanäle, sondern als Summen von **sphärischen Harmonischen**. Die erste Ordnung (First-Order Ambisonics, FOA) nutzt

vier Kanäle: einen omnidirektionalen „W-Kanal“ und drei gerichtete Komponenten (X/Y/Z). **Higher-Order Ambisonics (HOA)** erweitert die Ordnung n und damit die Zahl der Kanäle; für 3D-Ambisonics beträgt die Anzahl der B-Format-Komponenten $(n + 1)^2$, für eine 2D-Darstellung $2n + 1$ [20]. Die räumliche Auflösung wächst also quadratisch mit der Ordnung; ein dritter-Ordnung-Stream hat 16 Kanäle, eine fünfte Ordnung bereits 36 Kanäle. Jeder dieser Kanäle muss einzeln berechnet und gespeichert werden, was CPU- und Speicherbedarf quadratisch ansteigen lässt.

Trotz der vielen Kanäle sind die grundlegenden Operationen auf HOA-Signalen relativ günstig. Ein wichtiges Beispiel ist die **Rotation** des Schallfeldes, die nötig ist, wenn sich der Listener im Spiel dreht. Bspw. wird die Rotation durch eine Matrix-Multiplikation aller Ambisonics-Kanäle durchgeführt; die Entwickler betonen, dass der Aufwand pro Frame dem der Berechnung von Panning-Gains entspricht und „um Größenordnungen weniger anspruchsvoll“ als ein Filter oder eine Mischung ist [28]. Auch ein zeitvariabler „Emphasis Operator“, wie er in der Ambisonics-Forschung beschrieben wird, steigert die Ordnung mit einer kleinen Matrix-Multiplikation und benötigt nur „ein paar Multiplikationen pro Ausgabesample“ [10].

Bei der Dekodierung auf ein Lautsprecher-Set oder auf Kopfhörer (binaural) wird ein mehrkanaliger Ambisonics-Stream auf die benötigten Ausgabekanäle umgerechnet. Für Kopfhörer kommen wieder HRTFs zum Einsatz [20], wodurch die Komplexität der Binaural-Faltung (siehe Abschnitt 3.1) hinzukommt. Die Speicherkomplexität wird durch die Zahl der Ambisonics-Kanäle bestimmt; höhere Ordnungen liefern eine präzisere Lokalisation, reduzieren aber die Zahl der potentiellen Zuhörer (weil man viele Lautsprecher benötigt) und belasten die Rechenbudgets. Oft genügt FOA oder dritte Ordnung für Spiele, weil sie ein gutes Verhältnis von Präzision zu Ressourcen bieten.

3.3 Bildquellen-Methode und Ray-/Path-Tracing

Geometrische Verfahren simulieren die physikalische Ausbreitung von Schall im Level: Schallwellen werden von Wänden reflektiert, von Objekten abgeschattet oder gedämpft und legen reale Strecken zurück. Zwei grundsätzliche Ansätze sind verbreitet:

(a) Bildquellen-Methode (Image Source Method): Diese Methode spiegelt den Sender an jeder reflektierenden Oberfläche und generiert so virtuelle Quellen für jede Reflexion. Um eine erste Ordnung zu berechnen, müssen alle Flächen im Raum gespiegelt werden; für die zweite Ordnung werden alle diese ersten Bildquellen erneut an allen Flächen gespiegelt usw. Die Wayverb-Dokumentation fasst zusammen, dass die Zahl der notwendigen Tests sich als N^o ergibt, wobei N die Anzahl der Flächen und o der Reflexionsgrad ist; die Rechenzeit wächst also **exponentiell** [18]. Bei 100

Flächen dauert die Berechnung der zweiten Ordnung vielleicht eine Sekunde, die vierte Ordnung würde schon Stunden brauchen. Zudem müssen die meisten virtuellen Quellen als „invalide“ aussortiert werden, was zusätzlichen Verwaltungs- und Speicheraufwand bedeutet. Für einfache Räume und sehr frühe Reflexionen liefert diese Methode exakte Ergebnisse; bei komplexen Szenen oder langen Hallfahnen ist sie jedoch unpraktikabel.

(b) Ray-/Path-Tracing: Statt alle Spiegelquellen explizit zu generieren, werden viele Strahlen (Rays) in zufällige Richtungen ausgesendet. Jeder Strahl wird bei jedem Aufprall reflektiert oder gestreut; nur Strahlen, die den Hörer erreichen, tragen zur Impulsantwort bei. Wayverb weist darauf hin, dass die Komplexität der Ray-Tracing-Methode **linear** mit der Reflexionstiefe wächst was ein großer Vorteil gegenüber der exponentiellen Bildquellen-Methode mit sich bringt. In der Arbeit „Beam Tracing for Interactive Architectural Acoustics“ wird die erwartete Komplexität der Bildquellenmethode als $O(n^r)$ angegeben, wobei n die Anzahl der Flächen und r der Reflexionsgrad ist; für Ray-Tracing basiert der Aufwand lediglich auf Strahl-Flächen-Schnitt-Tests, die sublinear mit der Flächenzahl wachsen [5]. Ray-Tracing kann höhere Reflexionsordnungen schneller approximieren, leidet aber an „undersampling“: wichtige Pfade werden möglicherweise nicht getroffen, wenn zu wenige Strahlen eingesetzt werden. Dadurch ist das Ergebnis eher statistisch; das akustische Bild kann verrauscht sein, und die Präzision hängt von der Anzahl der Strahlen und der Qualität der Szene-Beschleunigungsstruktur ab.

Trade-off: Die Bildquellen-Methode bietet eine deterministische und physikalisch korrekte Lösung für frühe Reflexionen, ist aber bereits ab der dritten oder vierten Ordnung kaum noch interaktiv einsetzbar. Ray-Tracing ist wesentlich flexibler und kann auch Streuung oder Diffraction einbeziehen; allerdings müssen genügend Strahlen simuliert werden, um ein realistisches Ergebnis zu erhalten, was die Rechenlast wieder erhöht.

3.4 Geometrie-basiertes Delay- und Attenuation-Routing (Tapped-Delay-Line)

Eine vereinfachte geometrische Methode, die in Echtzeitumgebungen beliebt ist, nutzt **tapped delay lines**. Jedes reflektierte Signal wird als verzögerte, abgeschwächte Kopie des Originalsignals modelliert: Für eine reflektierte Strecke der Länge L werden $\frac{L \cdot f_c}{c}$ Samples Verzögerung in einen Puffer geschrieben, wobei f_c die Abtastrate und c die Schallgeschwindigkeit ist [19]. Der Tap wird mit $1/L$ gewichtet, um die Entfernung zu berücksichtigen. Diese Struktur kann problemlos für mehrere Reflexionen parallelisiert werden; die **Zeit- und Speicherkomplexität** pro Tap ist konstant, da nur eine Verzögerungsleitung und eine Multiplikation benötigt werden.

Allerdings steigt die Zahl der benötigten Taps schnell an, wenn man höherordige Reflexionen berücksichtigen möchte. Das DAFX-Paper zeigt, dass die Zahl der **virtuellen Quellen** N_{vir} , die man für frühe Reflexionen mit der Bildquellenmethode ermitteln müsste, exponentiell mit der maximalen Reflexionsordnung wächst, während die Zahl der tatsächlich sichtbaren Quellen N_{vis} mit der dritten Potenz der Zeit zunimmt. Aus diesem Grund werden tapped-delay-lines in der Praxis nur für die ersten beiden Reflexionen eingesetzt, während für den diffusen Nachhall andere Modelle (z. B. Feedback-Delay-Netzwerke) verwendet werden. Geometrie-basiertes Delay-Routing erlaubt bewegte Hörer mit minimaler Latenz, verzichtet jedoch auf komplexe Hall- und Diffusions-Effekte. Es eignet sich vor allem für Spiele, in denen Performance wichtiger ist als eine vollständig physikalische Abbildung.

Die folgende Tabelle bietet einen kompakten Vergleich der untersuchten Verfahren:

Verfahren	Zeitkomplexität (vereinfacht)	Speicher- /Kanalbedarf	Trade-offs
HRTF-basiertes binaurales Rendering	$O(N^2)$ pro Quelle bei direkter Faltung; $O(N \log N)$ mit FFT	Große HRTF-Bibliotheken (mehrere MB), Filterlängen 512–2048 Samples	Sehr präzise Lokalisation, aber rechenintensiv; personalisierte HRTFs erhöhen Aufwand
Ambisonics (FOA/HOA)	Rotation des Schallfelds per Matrixmultiplikation; Zahl der Kanäle $= (n + 1)^2$	Vier Kanäle bei FOA; 16 bei 3. Ordnung; 36 bei 5. Ordnung	Quadratisch steigender Kanalbedarf; Rotation günstig; Dekodierung auf Kopfhörer erfordert HRTF-Faltung
Bildquellen-Methode	$O(N^o)$ (exponentiell in Anzahl der Flächen N und Reflexionsordnung o)	Viele virtuelle Quellen; Verwaltung der gültigen Spiegelquellen nötig	Physikalisch exakte frühe Reflexionen; ab dritter/vierter Ordnung nicht mehr interaktiv nutzbar
Ray-/Path-Tracing	Linear in der Reflexions-Tiefe; sublinear zur Anzahl der Flächen	Speicher für Strahlen und Beschleunigungsstruktur	Flexibel (Streuung/Diffraktion möglich); Ergebnis statistisch, abhängig von Ray-Anzahl
Tapped-Delay-Line	Konstante Kosten pro Tap; Gesamtkosten steigen mit Anzahl der Taps	Verzögerungsleitung pro Tap	Extrem performant für einfache Reflexionen; kein diffuser Hall, schnell ungenau

3.5 Diskussion der Trade-offs

Die Wahl eines Spatial-Audio-Verfahrens hängt stark von der Zielplattform und dem gewünschten Immersionsgrad ab. **HRTF-Faltung** liefert über Kopfhörer die über-

zeugendsten Ergebnisse, ist aber rechenintensiv und benötigt große Datensätze; FFT-Optimierungen mildern, aber beseitigen dieses Problem nicht. **Ambisonics** ermöglicht flexible Lautsprecher- oder Kopfhörer-Wiedergabe und benötigt für höhere Ordnungen viele Kanäle; die Rechenkosten steigen quadratisch mit der Ordnung, wohingegen Operationen wie Rotation sehr leichtgewichtig sind. **Bildquellen-Methoden** liefern physikalisch genaue frühe Reflexionen, werden aber exponentiell teurer mit jeder zusätzlichen Wand und Reflexion. **Ray-/Path-Tracing** und **Beam-Tracing** umgehen diese Explosion, indem sie nur wahrscheinliche Pfade verfolgen; ihre Effizienz wächst linear mit der Reflexionstiefe, allerdings auf Kosten möglicher Samplingfehler. **Tapped-Delay-Lines** schließlich sind extrem performant und eignen sich gut für VR/AR-Systeme mit bewegten Hörern, verlieren aber rasch an Realismus, sobald komplexe Nachhallstrukturen gefragt sind.

Indem Entwickler diese Verfahren kombinieren (etwa HRTF-Faltung für nahe Quellen, HOA für Umgebungs-Ambienzen, Ray-Tracing für präzise frühe Reflexionen und Delay-Netzwerke für diffuse Hallanteile) lässt sich ein ausgewogenes Verhältnis zwischen akustischer Präzision und Laufzeit herstellen, das den Anforderungen moderner Computerspiele gerecht wird.

4 Engines und Middleware: Praxis-Check

Im vorigen Abschnitt wurden mehrere Verfahren gegenübergestellt. Für die Umsetzung im Labyrinth (Echtzeit, bewegte Begleitquellen wie Insekten/Mini-Bots) ist entscheidend, *welche* Engine *welche* Verfahren unter welchen API-, Lizenz- und Performance-Randbedingungen bereitstellt. Im Folgenden verknüpfe ich diese mit den *konkreten* Möglichkeiten in Unity (Fokus), Godot und Unreal und leite anschließend daraus die Brücke zur Implementierungsskizze ab.

4.1 Unity (Fokus)

4.1.1 Architektur und Verfahren

Unity besitzt *keinen* eingebauten binauralen Spatializer, sondern bietet eine *Audio Spatializer SDK* als Erweiterung des nativen Plug-in-SDKs; Spatialization wird pro AudioSource aktiviert und ersetzt den Standard-Panner durch die Implementierung des gewählten Plug-ins [25, 26, 27]. Unity unterstützt zudem das Abspielen von Ambisonics (AmbiX/FOA) inkl. Decoder-Pfad [24].

In der Praxis werden Drittanbieter-Spatializer ausgewählt (*Project Settings* → *Audio* → *Spatializer Plugin*) [26]. Typische Optionen (Auswahl):

- **Microsoft:** Historischer *MS HRTF Spatializer* (Kompatibilität mit älteren Windows XR/HoloLens) versus moderner *Microsoft Spatializer* (empfohlen für aktuelle Projekte) [23, 17].
- **Meta/Oculus:** ONSP/*Meta XR Audio* mit HRTF-Rendering, Ambisonics-Wiedergabe und *Audio Propagation* (Reflexionen/Occlusion) [13, 12, 14].
- **Steam Audio:** HRTF, Ambisonics-Spatialization/Rotation, Occlusion/Transmission, Reflexionen und Multi-Pfad-Schallausbreitung; Unity-Integration vorhanden [30, 29, 31].

4.1.2 API-Umfang (Unity-Sicht)

Die *Audio Spatializer SDK* stellt dem Plug-in Listener/Source-Daten (Position, Richtung, Matrix), Distanz, Winkel und Mixer-Zugriff bereit; die Aktivierung je Quelle erfolgt über das **Spatialize**-Flag, die Gewichtung zwischen 2D/3D über **spatialBlend** [25, 27]. Ambisonics werden clip-seitig markiert und über den Decoderpfad abgespielt [24]. Die Auswahl/Parametrisierung des konkreten Spatializers geschieht zentral in den Projekteinstellungen [26].

4.1.3 Lizenzmodell, Portabilität, typische Performance

Unity selbst hat keine Audio-Lizenzkosten; die rechtlichen Rahmenbedingungen ergeben sich aus dem gewählten Plug-in: Steam Audio (Apache-ähnlich, quelloffen), Meta/Microsoft (proprietär, kostenfrei). Plattformsupport variiert: Microsoft und Meta zielen u. a. auf Windows/Android; Steam Audio deckt Windows/macOS/Linux/Android/iOS ab [26, 29]. Performance-seitig gilt: binaurale HRTF-Spatialization benötigt mehr CPU als simples Panning; Meta dokumentiert zusätzliche Parameter und Warnhinweise (z. B. nur Mono-Quellen spatialisieren) [14]. Für Windows-XR empfiehlt Unity Learn, bei neuen Projekten den *Microsoft Spatializer* statt des alten MS-HRTF zu nutzen [23].

Unity-bezogene Trade-offs. (a) *Nahe, bewegte Begleiter (Mini-Bots):* aktivierte Spatialization nur für diese Quellen (anderenfalls Standard-Panning), kurze IRs/„leichtes“ HRTF-Preset im gewählten Plug-in. (b) *Ambiences/Umgebungsbedt:* Ambisonics-Clip (FOA) + Decoder [24]. (c) *Frühe Reflexionen/Occlusion:* Steam Audio-Pfad (Occlusion/Transmission, Ray-/Path-basierte Reflexionen) wenn benötigt [30, 29]. So bleibt das CPU-Budget planbar, ohne den räumlichen Eindruck nahe Quellen zu opfern.

4.2 Godot (kurz)

Godot bringt `AudioStreamPlayer3D` mit: positionsabhängige Wiedergabe inkl. *Distanz-Attenuation*, *Richtwirkung*, *Doppler*; für größere Distanzen wird automatisch ein *Low-Pass-Filter* angewandt (abschaltbar via Cutoff) [7, 6]. Godot ist *MIT-lizenziert* (keine Gebühren) [8]. Für HRTF/Akustik-Propagation existieren Community-Bindings zu Steam Audio; API-seitig erfolgt der Zugriff über `GDExtension/Nodes` (Projektabhängigkeit). Trade-off: sehr schlanke, portable Grundfunktion; fortgeschrittene Features via Plug-ins.

4.3 Unreal Engine (kurz)

Unreal unterscheidet nativ drei Spatialization-Wege: *Panning*, *Soundfield Spatialization* (Ambisonics/Sphärische Harmonische; höhere Ordnung = höhere Auflösung, mehr Rechenaufwand) und *Binaurale Spatialization* (HRTF) [3]. Ambisonics sind „native“ first-class Bürger der Audio-Engine [2]; zusätzlich erlaubt die C++-API Drittanbieter-Spatializer (z. B. Meta/Steam Audio) inkl. binauraler/HRTF- oder Soundfield-Pfade [1]. Für Meta existiert ein aktuelles Unreal-Spatializer-Plug-in [15, 16]. Lizenz: UE ist kostenlos bis zur Umsatzgrenze; Plugin-Lizenzen sind separat zu betrachten.

5 Gedankenexperiment: Implementierungsskizze für mein Labyrinth (Unity auf macOS)

Leitidee

Ich gestalte das Audio konsequent *Headphone-first*. Präzise Lokalisation brauche ich nur dort, wo der Spieler sie wirklich spürt: bei den kleinen, schwirrenden Begleitern. Für diese wähle ich binaurales Rendering (HRTF) mit schlanker Konfiguration; alles Übrige hält ein leises, räumliches Umgebungsbett zusammen (FOA/Ambisonics). Raumwirkung (Occlusion/Reflexionen) setze ich bewusst sparsam ein, damit das Laufzeitbudget stabil bleibt.

Rollen im Labyrinth

Ich kennzeichne wenige, klar getrennte Rollen:

- **Begleiter (Mini-Bots/Insekten):** sehr nahe, kleine Quellen, die den Avatar umkreisen.

- **Spot-Geräusche:** punktuelle Hinweise an Knotenpunkten die bei Wegfindung helfen.
- **Ambience:** ein leises, kontinuierliches Soundfield (FOA) als atmosphärischer Hintergrund für das gesamte Labyrinth.
- **NPC:** Zustände aus ÜB 2 (Idle/Walk/Run/Crawl/Hide) werden akustisch lesbar (Atmung, Schritte, Stoff).

Datenfluss und Einbindung

Ich lasse meine bestehende Datenpipeline aus ÜB 1 unverändert und *erweitere* sie nur um Audio-Metadaten weiter: Das Labyrinth-JSON erhält optionale Einträge für *Rolle* (Begleiter/Spot/Ambience) und *Position*. Der *LabyrinthManager* instanziert daraus Klangobjekte genauso, wie er heute Wände/Lichter instanziert. Dadurch bleibt die Struktur meines Projekts (eine zentrale Stelle, die den Level “setzt”) erhalten.

Verhalten

Begleiter kreisen mit kleiner, zufälliger Varianz um den Avatar (langsamer Drift statt hektischer Sprünge), damit die Außenortung stabil bleibt. Spot-Geräusche bleiben ortsfest. Die Ambience läuft als FOA-Bett in geringer Lautheit und füllt Lücken zwischen einzelnen Quellen. Der NPC mappt seine bestehenden Modi schlicht auf passende Cues (ruhige Atmung, Schritte-Tempo, Stoffreibung) und bleibt damit akustisch konsistent zu seinem Bewegungszustand.

Raumwirkung (macOS-freundlich und minimalinvasiv)

Ich halte zwei Stufen bereit:

1. **Leichtgewicht:** reine Sichtlinien-Heuristik (wenn Wand dazwischen, dann leiser und dumpfer). Diese Stufe funktioniert auf macOS ohne besondere Abhängigkeiten.
2. **Erweitert (falls verfügbar):** ein macOS-kompatibler Spatializer/Propagation-Pfad für Occlusion/erste Reflexionen *nur* in markanten Bereichen (enge Gänge, Ecken). Ich aktiviere ihn gezielt für wenige Quellen, damit das Budget nicht kippt.

Wichtig ist mir, dass die Nah-Präzision (Begleiter) nie zugunsten von viel “Hall” leidet.

Qualitätsstufen (Budget-Denke statt Zahlen)

Ich plane drei Profile, die ich je nach Gerät umschalten kann:

- **Low:** Ambience sehr leise, keine Raumwirkung, binaural nur auf den Begleitern.
- **Medium:** binaural auf Begleitern *und* einer wichtigen Spot-Quelle; einfache Occlusion.
- **High:** zusätzlich dezent mehr Raum (frühe Reflexionen in Teilbereichen), ohne die Anzahl aktiver binauraler Quellen stark zu erhöhen.

So bleibt das System auf meinem Mac stabil und vorhersehbar.

Testgedanke

Ich prüfe iterativ nur mit Kopfhörern: (1) können Spieler die Begleiter sicher lokalisieren (links/rechts/vorne/hinten/oben)? (2) lenken Spot-Geräusche an Weggabelungen sinnvoll? (3) fühlt sich Ambience angenehm an, ohne zu maskieren? Wenn (1) gut ist, skaliere ich behutsam Raumanteile hoch; wenn nicht, drehe ich sie herunter.

Warum genau diese Aufteilung?

Sie spiegelt meinen Trade-off: *maximale Präzision dort, wo sie wirklich zählt* (wenige Nahquellen), *breite Räumlichkeit mit minimalem Overhead* (FOA) und *nur so viel Geometrie, wie atmosphärisch nötig ist*. Außerdem passt sie zu meinem bestehenden Client: Der *LabyrinthManager* bleibt die einzige Schaltstelle; der NPC aus ÜB 2 bleibt unverändert und wird nur akustisch “lesbar” gemacht. Für macOS vermeide ich harte Abhängigkeiten und halte ein sauberes Fallback bereit.

Literatur

- [1] Epic Games. *Audio Engine Overview in Unreal Engine*. Drittanbieter-Plugins (HRTF/Ambisonics) via C++-API. 2025. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/audio-engine-overview-in-unreal-engine> (besucht am 09.09.2025).
- [2] Epic Games. *Native Soundfield Ambisonics Rendering in Unreal Engine*. Ambisonics als native Engine-Funktion. 2025. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/native-soundfield-ambisonics-rendering-in-unreal-engine> (besucht am 09.09.2025).

- [3] Epic Games. *Spatialization Overview in Unreal Engine*. Panning, Soundfield (Ambisonics), Binaural (HRTF). 2025. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/spatialization-overview-in-unreal-engine> (besucht am 09.09.2025).
- [4] Interaction Design Foundation. *Spatial Audio*. Aug. 2025. URL: <https://www.interaction-design.org/literature/topics/spatial-audio> (besucht am 14.09.2025).
- [5] Thomas Funkhouser, Nicolas Tsingos, Ingrid Carlbom, Gary Elko, Mohan Sondhi, James E. West, Gopal Pingali, Patrick Min und Addy Ngan. „A Beam Tracing Method for Interactive Architectural Acoustics“. In: *The Journal of the Acoustical Society of America* 115.2 (2004), S. 739–756.
- [6] Godot Documentation. *AudioStreamPlayer3D (3.5) — auto Low-Pass & Cutoff*. Abschaltung des Low-Pass via Cutoff=20500 Hz. 2022. URL: https://docs.godotengine.org/en/3.5/classes/class_audiostreamplayer3d.html (besucht am 07.09.2025).
- [7] Godot Documentation. *AudioStreamPlayer3D (latest)*. Distanz-Attenuation, Richtwirkung, Doppler, automatischer Low-Pass. 2025. URL: https://docs.godotengine.org/en/latest/classes/class_audiostreamplayer3d.html (besucht am 07.09.2025).
- [8] Godot Engine. *Godot License (MIT)*. MIT-Lizenz, keine Gebühren. 2025. URL: <https://godotengine.org/license/> (besucht am 07.09.2025).
- [9] Pavel Karas und David Svoboda. „Algorithms for Efficient Computation of Convolution“. In: *Design and Architectures for Digital Signal Processing*. Rijeka, Croatia: IntechOpen, 2013, S. 179–208.
- [10] W. Bastiaan Kleijn. „Directional Emphasis in Ambisonics“. In: *IEEE Signal Processing Letters* 25.7 (2018), S. 1079–1083.
- [11] Aimee L. Lalime, Marty E. Johnson und Stephen A. Rizzi. *Development of an Efficient Binaural Simulation for the Analysis of Structural Acoustic Data*. Techn. Ber. NASA Langley Research Center, 2002.
- [12] Meta Horizon OS Developers. *Oculus Spatializer Features*. Feature-Übersicht inkl. Ambisonics/Dynamic Room Modeling. 2024. URL: <https://developers.meta.com/horizon/documentation/unity/audio-spatializer-features/> (besucht am 11.09.2025).

- [13] Meta Horizon OS Developers. *Oculus Spatializer Plugin for Unity — Requirements and Setup*. Unity-Integration, Audio Propagation (Reverb/Occlusion). 2024. URL: <https://developers.meta.com/horizon/documentation/unity/audio-osp-unity-req-setup/> (besucht am 11.09.2025).
- [14] Meta Horizon OS Developers. *Apply Spatialization in Unity (Meta XR Audio SDK)*. Aktualisierte Anleitung und Parameter, Stand 2025. 2025. URL: <https://developers.meta.com/horizon/documentation/unity/meta-xr-audio-sdk-unity-spatialize/> (besucht am 12.09.2025).
- [15] Meta Horizon OS Developers. *Apply Spatialization in Unreal (Meta XR Audio SDK)*. Unreal-spezifische Spatializer-Integration. 2025. URL: <https://developers.meta.com/horizon/documentation/unreal/meta-xr-audio-sdk-unreal-spatialize/> (besucht am 15.09.2025).
- [16] Meta Horizon OS Developers. *Meta XR Audio SDK for Unreal — Features*. HRTF-basierte Spatialization in Unreal. 2025. URL: <https://developers.meta.com/horizon/documentation/unreal/meta-xr-audio-sdk-unreal-features/> (besucht am 15.09.2025).
- [17] Microsoft Learn. *Spatial sound in Unity*. Einrichtung des Microsoft Spatializer in Unity. 2023. URL: <https://learn.microsoft.com/en-us/windows/mixed-reality/develop/unity/spatial-sound-in-unity> (besucht am 11.09.2025).
- [18] Reuk. *Wayverb: Image-Source Model*. 2020. URL: https://reuk.github.io/wayverb/image_source.html (besucht am 06.09.2025).
- [19] Augusto Sarti, Stefano Tubaro u. a. „Low-cost geometry-based acoustic rendering“. In: *Proceedings of COST-G6 Conference on Digital Audio Effects (DAFx01), Limerick (Iriska)*. Bd. 6. 8. 2001, S. 19–22.
- [20] Jan Skorupa und Maciej Głowiak. *Guidelines: Ambisonic Audio Production 1.0*. Techn. Ber. Ars Electronica Futurelab, 2023. URL: <https://ars.electronica.art/futurelab/files/2023/04/guidelines-ambisonic-audio-production-1.0.pdf> (besucht am 03.09.2025).
- [21] *Spatial Audio: The Continuing Evolution*. Abbey Road Institute. 2023. URL: <https://abbeyroadinstitute.nl/blog/spatial-audio-continuing-evolution/> (besucht am 31.08.2025).
- [22] *Spatialization Overview in Unreal Engine*. Epic Games. 2025. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/spatialization-overview-in-unreal-engine> (besucht am 01.09.2025).

- [23] Unity Learn. *Best Practices for Immersive Audio in VR*. Hinweis auf zwei Microsoft-Spatializer (MS HRTF vs. Microsoft Spatializer). 2024. URL: <https://learn.unity.com/tutorial/best-practices-for-immersive-audio-in-vr> (besucht am 11.09.2025).
- [24] Unity Technologies. *Ambisonic Audio — Unity Manual*. Ambisonics/Decoderpfad in Unity. 2020. URL: <https://docs.unity3d.com/2020.1/Documentation/Manual/AmbisonicAudio.html> (besucht am 14.09.2025).
- [25] Unity Technologies. *Audio Spatializer SDK - Unity Manual*. SDK-Übersicht und API-Kontext. 2020. URL: <https://docs.unity3d.com/Manual/AudioSpatializerSDK.html> (besucht am 14.09.2025).
- [26] Unity Technologies. *Audio spatializers in XR — Unity Manual*. Kein Built-in-Spatializer; Auswahl Drittanbieter-Plug-ins. 2024. URL: <https://docs.unity3d.com/2023.2/Documentation/Manual/VRAudioSpatializer.html> (besucht am 14.09.2025).
- [27] Unity Technologies. *AudioSource.spatialBlend — Scripting API*. 2D/3D-Gewichtung und Bezug zu Attenuation/Doppler. 2024. URL: <https://docs.unity3d.com/ScriptReference/AudioSource-spatialBlend.html> (besucht am 14.09.2025).
- [28] *Using Ambisonics for Dynamic Ambiences*. Audiokinetic Inc. 2016. URL: <https://www.audiokinetic.com/en/blog/using-ambisonics-for-dynamic-ambiences/> (besucht am 04.09.2025).
- [29] Valve Software. *Steam Audio — Steamworks Documentation*. Featureliste, Plattformen. 2025. URL: https://partner.steamgames.com/doc/features/steam_audio (besucht am 12.09.2025).
- [30] Valve Software. *Steam Audio Unity Integration — Documentation*. HRTF (SOFA), Ambisonics, Occlusion/Transmission, Reflexion/Propagation. 2025. URL: <https://valvesoftware.github.io/steam-audio/doc/unity/index.html> (besucht am 12.09.2025).
- [31] Valve Software. *User's Guide — Steam Audio Unity Integration*. Ambisonics in Unity, FOA/Clip-Workflow. 2025. URL: <https://valvesoftware.github.io/steam-audio/doc/unity/guide.html> (besucht am 12.09.2025).